

tryLock

Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

Associé à un ReentrantLock

lock

-  Bloquant quoi qu'il arrive

tryLock

-  rend une valeur si le verrou ne peut être pris
-  Deux formes
 - ▶ `boolean tryLock()`
 - ▶ `boolean tryLock(long time, TimeUnit unit)`

Par l'exemple

3

LeParking (avec des ReentrantLock)

 En remplacement des synchronized + Object

 On va «jouer» avec un tryLock

- ▶ Exemple un peu artificiel...
- ▶ Sera revu en TD/TME

Nouveautés

4

```
import java.util.concurrent.locks.*;

@SuppressWarnings("serial") // Eviter le warning sur MonException
class MonException extends Exception {
    MonException (String m) {
        super (m);
    }
}
```


La classe Automobiliste revisitée

5

```
class Automobiliste implements Runnable {  
    private static int cpt = 1;  
    private static final Object mutex = new Object();  
    private int monId;  
    private Parking p;  
  
    Automobiliste (Parking p) {...}  
    private void trace (String m) {...}  
  
    public void run() {  
        try {  
            this.trace ("je rentre");  
            p.entrer (monId);  
            this.trace ("je me ballade");  
            p.sortir(monId);  
            this.trace ("je sort");  
        } catch (Exception e) {  
            this.trace ("j'ai pris " + e); // en cas de souci  
        }  
    }  
}
```


La classe Parking revisitée

6

```
class Parking {
    private int capacite;
    private final ReentrantLock l = new ReentrantLock(); // nouveau
    private final Condition file = l.newCondition(); // nouveau

    Parking (int taille) {...}

    public void entrer (int auto) throws MonException, InterruptedException {
        if (l.tryLock()) { // on ne supporte pas d'attendre du tout!
            while (capacite == 0) { // toujours le signalAll
                System.out.println (" " + auto + " en attente");
                file.await();
            }
            capacite--;
            System.out.println (" " + auto + " dans le parking");
            l.unlock();
        } else {
            throw new MonException("verrou occupe"); // pour un exemple
        }
    }

    public void sortir (int auto) {
        l.lock();
        capacite++;
        System.out.println (" " + auto + " sort");
        file.signalAll();
        l.unlock();
    }
}
```


Une exécution...

7

```
$ java LeParking2 3 6
Automobiliste 1 : je rentre
Automobiliste 5 : je rentre
Automobiliste 4 : je rentre
Automobiliste 3 : je rentre
Automobiliste 2 : je rentre
Automobiliste 3 : j'ai pris MonException: verrou occupe
Automobiliste 4 : j'ai pris MonException: verrou occupe
Automobiliste 5 : j'ai pris MonException: verrou occupe
  1 dans le parking
Automobiliste 6 : je rentre
Automobiliste 1 : je me ballade
Automobiliste 2 : j'ai pris MonException: verrou occupe
  6 dans le parking
Automobiliste 6 : je me ballade
  1 sort
Automobiliste 1 : je sort
  6 sort
Automobiliste 6 : je sort
```


En guise de conclusion...

8

Permet d'éviter un blocage

-  Attente active (ou alternative d'exécution)
-  Attente semi-active
-  etc.

Complément indispensable du lock()/unlock()

